

PROPOSAL TUGAS AKHIR

**CONFIGURATION DRIFT PADA DEPLOYMENT APLIKASI
MULTI-SERVICE DI KLASSTER KUBERNETES: EVALUASI EMPIRIS
PARADIGMA MANUAL DAN GITOPS (STUDI KASUS INVENIORDM)**

***CONFIGURATION DRIFT IN MULTI-SERVICE APPLICATION
DEPLOYMENTS ON KUBERNETES CLUSTERS: AN EMPIRICAL
EVALUATION OF MANUAL AND GITOPS PARADIGMS (A CASE
STUDY OF INVENIORDM)***



MUHAMMAD ARGYA VITYASY
23/522547/PA/22475

**PROGRAM STUDI ILMU KOMPUTER
DEPARTEMEN ILMU KOMPUTER DAN ELEKTRONIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS GADJAH MADA
YOGYAKARTA**

2026

PROPOSAL TUGAS AKHIR

**CONFIGURATION DRIFT PADA DEPLOYMENT APLIKASI
MULTI-SERVICE DI KLASSTER KUBERNETES: EVALUASI EMPIRIS
PARADIGMA MANUAL DAN GITOPS (STUDI KASUS INVENIORDM)**

***CONFIGURATION DRIFT IN MULTI-SERVICE APPLICATION
DEPLOYMENTS ON KUBERNETES CLUSTERS: AN EMPIRICAL
EVALUATION OF MANUAL AND GITOPS PARADIGMS (A CASE
STUDY OF INVENIORDM)***

Usulan Penelitian



MUHAMMAD ARGYA VITYASY
23/522547/PA/22475

**PROGRAM STUDI ILMU KOMPUTER
DEPARTEMEN ILMU KOMPUTER DAN ELEKTRONIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS GADJAH MADA
YOGYAKARTA**

2026

HALAMAN PENGESAHAN

PROPOSAL TUGAS AKHIR

CONFIGURATION DRIFT PADA DEPLOYMENT APLIKASI MULTI-SERVICE DI KLASER KUBERNETES: EVALUASI EMPIRIS PARADIGMA MANUAL DAN GITOPS (STUDI KASUS INVENIORDM)

Telah dipersiapkan dan disusun oleh

MUHAMMAD ARGYA VITYASY
23/522547/PA/22475

Telah dipertahankan di depan Tim Penguji
pada tanggal 16 Juni 2026

Susunan Tim Penguji

Guntur Budi Herwanto, S.Kom., M.Cs. Pembimbing	Nama Dosen Penguji 1 Ketua Penguji
--	---

Nama Dosen Penguji 2
Anggota Penguji

DAFTAR ISI

Halaman Judul	ii
Halaman Pengesahan	iii
INTISARI	vii
ABSTRACT	viii
I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Batasan Masalah	3
1.4 Tujuan Penelitian	3
1.5 Manfaat Penelitian	4
1.6 Sistematika Penulisan	4
II PENELITIAN TERKAIT	6
2.1 <i>Infrastructure Configuration Management</i> dan <i>Configuration Drift</i> . .	6
2.1.1 Faktor yang Menimbulkan Configuration Drift	7
2.2 <i>Continuous Deployment</i> dan Paradigma GitOps	7
2.3 Dasar Teori: <i>Reconciliation</i> dan <i>Convergence</i> pada Sistem Deklaratif .	8
2.3.1 <i>Convergence Menuju Desired State</i>	8
2.3.2 <i>Audit Trail</i> sebagai <i>Social Control Mechanism</i>	9
2.4 Evaluasi Empiris GitOps dan ArgoCD	9
2.5 Metode Eksperimental Terkontrol pada Riset Sistem	10
2.6 InvenioRDM sebagai Beban Uji (Workload)	11
2.7 Kesenjangan Penelitian (<i>Research Gap</i>)	11
III METODE DAN RANCANGAN PENELITIAN	13
3.1 Deskripsi Umum Penelitian	13
3.2 Metodologi	15
3.3 Definisi Intervensi dan Variabel Penelitian	16
3.3.1 Intervensi (Perlakuan)	16
3.3.2 Variabel Independen (Variabel Bebas)	17

3.3.3	Variabel Dependen (Variabel Terikat)	17
3.4	Peran Peneliti dan Standarisasi Prosedur	19
3.5	Skrip Otomasi Eksperimen	20
3.5.1	Skrip <i>Drift Injection</i> (<code>drift-inject.sh</code>)	20
3.5.2	Skrip Pengumpulan Metrik (<code>metrics-collect.sh</code>)	20
3.5.3	Skrip Verifikasi Baseline (<code>baseline-check.sh</code>)	21
3.5.4	Skrip Recovery Runbook (<code>recovery-runbook.sh</code>)	21
3.6	Lingkungan Eksperimen	21
3.6.1	Klaster Kubernetes	21
3.6.2	ArgoCD (Lingkungan B)	22
3.6.3	Beban Uji (Workload)	22
3.6.4	Kontrol Variabel Pengganggu	22
3.6.5	Protokol <i>Monitoring</i> Variabel Terkontrol	23
3.7	Analisis Power dan Justifikasi Ukuran Sampel	23
3.8	Matriks Eksperimen	24
3.9	Skenario Drift Terkontrol	24
3.9.1	Skenario A: Perubahan Replika (Replica Drift)	24
3.9.2	Skenario B: Perubahan Image (Image Drift)	25
3.9.3	Skenario C: Perubahan ConfigMap (ConfigMap Drift)	25
3.9.4	Skenario D: <i>Resource Deletion</i>	25
3.9.5	Skenario E: Patch Manual Tidak Sah (Unauthorized Manual Patch)	25
3.9.6	Skenario F: Drift Lintas Namespace (Cross-namespace Drift)	25
3.9.7	Skenario G: Perubahan Secret (Secret Drift)	25
3.10	Protokol Pengukuran	26
3.10.1	Protokol Pengukuran Baseline (O1)	26
3.10.2	Protokol <i>Drift Injection</i> (X)	26
3.10.3	Protokol Pengukuran Pasca- <i>Injection</i> (O2)	26
3.11	Metode Pengujian (Eksperimen E1 s.d. E3)	27
3.11.1	E1: Konsistensi Deployment (R1)	27
3.11.2	E2: Waktu Deteksi, Pemulihan, dan Identifikasi Akar Masalah (R2)	27
3.11.3	E3: <i>Traceability</i> Operasional (R3, Deskriptif)	28
3.12	Analisis Statistik Komparatif	28
3.13	Ancaman terhadap Validitas	29

3.13.1 Validitas Internal	29
3.13.2 Validitas Eksternal	30
3.13.3 Validitas Konstruk	30
3.13.4 Reliabilitas	30
IV JADWAL PENELITIAN	32
4.1 Penjelasan Kegiatan	32

INTISARI

Configuration Drift pada Deployment Aplikasi Multi-Service di Klaster Kubernetes: Evaluasi Empiris Paradigma Manual dan GitOps (Studi Kasus InvenioRDM)

Oleh

Muhammad Argya Vityasy

23/522547/PA/22475

Deployment aplikasi multi-service pada klaster Kubernetes menggunakan pendekatan manual sering menimbulkan *configuration drift* akibat intervensi manusia, yang mengakibatkan *inconsistency state* klaster, *audit trail* yang buruk, dan *operational recovery time* yang lama. Penelitian ini mengevaluasi secara empiris dampak skenario *configuration drift* terhadap karakteristik operasional deployment pada dua paradigma manajemen yang berbeda, yaitu manual dan GitOps, pada klaster Kubernetes dengan studi kasus InvenioRDM. Penelitian mengadopsi desain eksperimental terkontrol dengan dua lingkungan paralel: Lingkungan A (deployment manual) dan Lingkungan B (deployment GitOps). Pada kedua lingkungan, *configuration drift* di-inject secara terkontrol melalui tujuh skenario (perubahan replika, perubahan *image*, modifikasi ConfigMap, *resource deletion*, *patch* manual, *drift* lintas *namespace*, dan perubahan Secret). Pengukuran dilakukan terhadap empat metrik utama: (1) konsistensi deployment; (2) *detection time* dan *drift recovery time*; (3) jumlah intervensi manual pasca-drift; dan (4) tingkat *audit trail* (*traceability*). Setiap skenario diulang minimal lima kali per lingkungan untuk menghasilkan data yang memadai. Hasil dianalisis menggunakan uji statistik komparatif (uji normalitas Shapiro–Wilk, uji perbandingan *independent samples t-test* atau *Mann–Whitney U*, dan ukuran efek Cohen’s d atau Cliff’s δ) untuk mengkarakterisasi perbedaan antara kedua paradigma. Hasil penelitian ini diharapkan memberikan bukti empiris mengenai dampak operasional skenario *configuration drift* pada lingkungan Kubernetes.

ABSTRACT

Configuration Drift in Multi-Service Application Deployments on Kubernetes Clusters: An Empirical Evaluation of Manual and GitOps Paradigms (A Case Study of InvenioRDM)

By

Muhammad Argya Vityasy

23/522547/PA/22475

Manual deployment of multi-service applications on Kubernetes clusters frequently introduces configuration drift due to human intervention, resulting in inconsistent cluster state, poor audit trails, and lengthy operational recovery time. This research empirically evaluates the impact of configuration drift scenarios on deployment operational characteristics under two management paradigms, namely manual and GitOps, on Kubernetes clusters, using InvenioRDM as a case study. The study adopts a controlled experimental design with two parallel environments: Environment A (manual deployment) and Environment B (GitOps deployment). On both environments, controlled configuration drift is injected through seven scenarios (replica change, image change, ConfigMap modification, resource deletion, manual patch, cross-namespace drift, and Secret change). Measurements are taken across four primary metrics: (1) deployment consistency; (2) drift detection and recovery time; (3) manual intervention count post-drift; and (4) operational traceability. Each scenario is repeated at least five times per environment to yield sufficient data. Results are analyzed using comparative statistical tests (Shapiro–Wilk normality test, independent samples t-test or Mann–Whitney U comparison, and Cohen’s d or Cliff’s delta effect sizes) to characterize the differences between the two paradigms. The results of this study are expected to provide empirical evidence on the operational impact of configuration drift scenarios in Kubernetes environments.

BAB I

PENDAHULUAN

1.1 Latar Belakang

Kubernetes telah menjadi platform *container orchestration* yang dominan di industri maupun akademisi, digunakan oleh lebih dari 5,6 juta developer secara global [Cloud Native Computing Foundation, 2024]. Kemampuan Kubernetes dalam mengelola deployment, *scaling*, dan *self-healing* menjadikannya pilihan utama untuk menjalankan aplikasi yang memerlukan *availability* tinggi [Burns et al., 2016]. Namun, seiring bertambahnya jumlah layanan yang di-deploy pada sebuah klaster, kompleksitas operasional meningkat secara drastis. Sebuah aplikasi sekompleks Invenio-RDM, yaitu platform repositori data penelitian *open-source* yang dikembangkan oleh CERN, terdiri dari setidaknya 16 komponen yang saling bergantung: layanan *web*, *worker* asinkron, database relasional, *search engine*, *cache*, *object storage*, *ingress controller*, *certificate manager*, enkripsi *secret*, tunnel jaringan, kebijakan keamanan, *monitoring system*, dan *backup* [CERN, 2024].

Pengelolaan deployment aplikasi multi-service menggunakan pendekatan manual, yakni melalui perintah `kubectl apply`, *manual scaling*, atau *patching* langsung pada klaster, sering menghasilkan *inconsistency* operasional. Praktik ini rentan terhadap *configuration drift*, yaitu situasi di mana *state* aktual klaster menyimpang dari *state* yang didefinisikan secara deklaratif [Zhang et al., 2026]. Drift dapat timbul akibat tiga penyebab utama: (1) *perubahan langsung pada klaster* yang tidak tercatat dalam sistem *version control*; (2) *prosedur deployment yang tidak konsisten* antar operator; dan (3) *absence of automatic detection mechanisms* yang dapat mengidentifikasi deviasi sebelum menimbulkan insiden.

Penelitian empiris oleh Zhang et al. [2026] menunjukkan bahwa 71% dari 2.260 skrip Kubernetes yang dianalisis mengandung *misconfiguration* yang bermakna, di mana sebagian besar menyebabkan *downtime* atau degradasi layanan. Rahman et al. [2023] menambahkan bahwa *security misconfiguration* pada manifest Kubernetes mencakup 11 kategori yang berulang, termasuk *container* yang berjalan sebagai root dan tanpa *resource limits*. Studi-studi ini mengonfirmasi bahwa drift bukan sekadar masalah estetika konfigurasi, melainkan ancaman nyata terhadap keandalan dan keamanan layanan.

GitOps telah muncul sebagai paradigma yang diajukan sebagai alternatif untuk mengatasi masalah ini dengan menjadikan repositori Git sebagai *single source of truth* dan mengandalkan agen *software* untuk menyelaraskan *state* kluster secara otomatis [CNCF GitOps Working Group, 2021]. Beberapa penelitian mendukung klaim ini secara kualitatif: Nataraja [2025] menemukan bahwa pendekatan deklaratif menghasilkan *drift correction* yang lebih cepat dan paparan kerentanan yang lebih rendah; Shrestha and Ali [2024] melaporkan karakteristik GitOps pada aspek *drift detection*; dan Matubber [2025] mengukur *reconciliation time* GitOps terhadap *self-healing*. Namun, temuan-temuan ini mengandung kelemahan metodologis, yakni tidak ada yang mengisolasi efek paradigma deployment dari variabel pengganggu, dan tidak ada yang menerapkan uji statistik formal pada perbandingan terkontrol [Shrestha and Ali, 2024, Nataraja, 2025, Matubber, 2025].

Kesenjangan ini menjadi landasan penelitian ini: terdapat kebutuhan untuk mengevaluasi secara empiris seberapa besar dampak *configuration drift* terhadap karakteristik operasional deployment (konsistensi, *detection time/recovery time*, dan *traceability*) ketika berbagai skenario drift terjadi pada dua paradigma manajemen deployment yang berbeda, yaitu manual dan GitOps, melalui evaluasi empiris terkontrol pada sebuah *workload* multi-service (InvenioRDM) di kluster Kubernetes.

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, penelitian ini merumuskan permasalahan utama sebagai berikut: sejauh mana paradigma manajemen deployment yang berbeda, yaitu pendekatan manual menggunakan `kubectl` dan pendekatan GitOps menggunakan ArgoCD, menghasilkan perbedaan pada karakteristik operasional ketika menghadapi *configuration drift* yang terkontrol? Permasalahan ini diuraikan ke dalam tiga dimensi pengukuran. Pertama, sejauh mana konsistensi deployment berbeda antara kedua paradigma setelah terjadinya drift? Kedua, seberapa besar perbedaan *detection time*, *recovery time*, dan *root cause identification time* antara lingkungan yang mendeteksi dan memulihkan secara otomatis dan lingkungan yang bergantung pada intervensi manual? Ketiga, sejauh mana perbedaan *traceability* operasional antara paradigma yang mencatat setiap perubahan secara otomatis melalui Git dan paradigma yang mengandalkan *manual logging*?

Untuk menjawab pertanyaan-pertanyaan tersebut, kedua lingkungan diuji terhadap tujuh skenario *configuration drift* yang merepresentasikan kesalahan opera-

sional umum pada Kubernetes: (A) perubahan replika yang melewati deklarasi Git, (B) perubahan *image* kontainer langsung pada klaster, (C) modifikasi ConfigMap tanpa pembaruan *source of truth*, (D) *resource deletion* secara manual, (E) *patch* yang tidak sah pada sumber daya yang dikelola, (F) aplikasi sumber daya pada *namespace* yang salah, dan (G) perubahan Secret di luar Git.

1.3 Batasan Masalah

Agar penelitian ini tetap fokus dan terukur, batasan masalah yang ditetapkan adalah:

1. Penelitian dilakukan pada klaster Kubernetes tunggal yang dikelola Rancher dengan spesifikasi 3 node, 24 total inti CPU, dan ~ 23 Gi total memori; skenario *multi-cluster* tidak termasuk dalam ruang lingkup.
2. Studi kasus yang digunakan adalah deployment InvenioRDM beserta 16 komponen infrastruktur dan dependensinya. InvenioRDM merupakan subjek uji (*test subject*), bukan objek penelitian.
3. Skenario *drift* yang diuji terbatas pada tujuh kategori: (A) perubahan replika, (B) perubahan *image*, (C) modifikasi ConfigMap, (D) *resource deletion*, (E) *patch* manual tidak sah, (F) drift lintas *namespace*, dan (G) perubahan Secret.
4. Dua paradigma manajemen deployment dibandingkan: (a) deployment manual menggunakan `kubectl`, dan (b) deployment GitOps menggunakan ArgoCD.
5. Analisis statistik menggunakan *Mann–Whitney U test* untuk uji signifikansi perbedaan antara kedua lingkungan, dengan tingkat signifikansi $\alpha = 0,05$ dan ukuran efek (Cohen's d atau Cliff's δ).
6. Paradigma perantara (Git sebagai *source of truth* tanpa agen *reconciliation* otomatis) tidak termasuk dalam ruang lingkup penelitian ini.

1.4 Tujuan Penelitian

Tujuan dari penelitian ini adalah:

1. Mengukur dan membandingkan konsistensi deployment pada lingkungan GitOps dan lingkungan manual setelah skenario *configuration drift* terkontrol.

2. Mengukur dan membandingkan *detection time*, *recovery time*, dan *root cause identification time* antara kedua lingkungan.
3. Mendokumentasikan dan membandingkan *traceability* operasional antara lingkungan GitOps dan manual secara deskriptif.

1.5 Manfaat Penelitian

Penelitian ini diharapkan memberikan manfaat berikut:

1. **Teoritis:** Memberikan bukti empiris terukur mengenai dampak operasional skenario *configuration drift* pada dua paradigma deployment, yang selama ini banyak didasarkan pada klaim vendor atau studi kasus deskriptif. Hasil ini melengkapi literatur akademis dengan data eksperimental yang dapat direproduksi dan diverifikasi.
2. **Metodologis:** Menawarkan protokol eksperimental yang dapat direplikasi, termasuk skenario drift terstandarisasi, metrik operasional, dan prosedur analisis statistik, sehingga penelitian selanjutnya dapat mereplikasi, memperluas, atau mengkritik temuan ini pada konteks dan platform yang berbeda.
3. **Praktis:** Menyediakan wawasan operasional yang dapat membantu administrator kluster Kubernetes dalam memilih paradigma deployment yang sesuai berdasarkan bukti kuantitatif, serta menyoroti skenario drift yang paling rentan pada masing-masing paradigma.

1.6 Sistematika Penulisan

Sistematika penulisan proposal tugas akhir ini adalah sebagai berikut:

- **Bab I:** Pendahuluan, berisi latar belakang, rumusan masalah, batasan masalah, tujuan penelitian, manfaat penelitian, dan sistematika penulisan.
- **Bab II:** Penelitian Terkait, membahas studi empiris terkait *configuration drift* pada Kubernetes, evaluasi GitOps, dasar teori *reconciliation* dan *convergence*, metode eksperimental pada riset sistem, serta *research gap* yang menjadi landasan penelitian.

- **Bab III:** Metode dan Rancangan Penelitian, menjelaskan metodologi evaluasi empiris terkontrol, definisi variabel dan intervensi, tujuh skenario drift, protokol pengukuran, analisis statistik komparatif, ancaman terhadap validitas, dan lingkungan eksperimen.
- **Bab IV:** Jadwal Penelitian, memuat Gantt Chart berbasis bulan untuk perencanaan waktu penelitian selama 8 bulan.

BAB II

PENELITIAN TERKAIT

2.1 *Infrastructure Configuration Management dan Configuration Drift*

Infrastructure state management adalah disiplin yang mengatur bagaimana *state* sebuah sistem infrastruktur didefinisikan, diterapkan, dan dipertahankan. Dalam paradigma deklaratif, operator mendefinisikan *desired state* melalui konfigurasi yang telah dibakukan (*Infrastructure-as-Code*), dan sistem bertanggung jawab untuk mencapai serta mempertahankan *state* tersebut. Dalam paradigma imperatif, operator memberikan serangkaian perintah langkah demi langkah yang mengubah *state* sistem secara langsung [Limoncelli et al., 2018]. Perbedaan fundamental ini memiliki implikasi langsung terhadap bagaimana sistem merespons deviasi konfigurasi.

Configuration drift adalah situasi di mana *state* aktual sebuah sistem menyimpang dari *state* yang didefinisikan secara deklaratif [Pohjola, 2025]. Dalam konteks Kubernetes, drift terjadi ketika operator, baik secara langsung maupun tidak sengaja, mengubah manifest kluster tanpa memperbarui *source of truth* deklaratif, sehingga *state* aktual dan *desired state* menjadi tidak konsisten.

Penelitian empiris oleh Zhang et al. [2026] melakukan studi terhadap 719 defect konfigurasi dari 2.260 skrip Kubernetes dan mengidentifikasi 15 kategori defect, di mana 71% di antaranya menyebabkan *downtime* atau degradasi layanan. Studi ini menegaskan bahwa *misconfiguration* merupakan risiko operasional utama pada Kubernetes. Rahman et al. [2023] menambahkan bahwa *security misconfiguration* pada manifest Kubernetes *open-source* mencakup 11 kategori yang berulang, termasuk *container* yang berjalan sebagai root, tanpa *drop capabilities*, dan tanpa *resource limits*. Penelitian menunjukkan bahwa drift bukan hanya masalah estetika konfigurasi, tetapi ancaman nyata terhadap keandalan dan keamanan layanan.

Pohjola [2025] memberikan kerangka konseptual untuk memahami sumber dan dampak drift pada sistem *Infrastructure-as-Code* (IaC). Penelitian ini mengidentifikasi tiga faktor pendorong utama drift: (1) *human expertise* yang tidak konsisten; (2) *time pressure* yang memicu perubahan cepat dan tidak didokumentasikan; dan (3) *absence of automatic detection mechanisms* pada pipeline deployment. Meskipun penelitian ini tidak beroperasi pada Kubernetes secara eksklusif, prinsip-prinsip yang dikemukakan berlaku umum untuk platform orkestrasi berbasis konfigurasi deklara-

tif.

2.1.1 Faktor yang Menimbulkan Configuration Drift

Berdasarkan literatur, faktor-faktor utama yang memperburuk *configuration drift* pada Kubernetes meliputi:

1. **Perubahan langsung pada klaster:** Penggunaan `kubectl edit`, `kubectl patch`, atau `kubectl scale` secara langsung tanpa memperbarui repositori *source of truth* [Zhang et al., 2026].
2. **Prosedur deployment yang tidak terdokumentasi:** Setiap operator dapat memiliki urutan perintah yang berbeda, yang mengakibatkan *state* klaster tidak deterministik [Pohjola, 2025].
3. **Kurangnya mekanisme *rollback* yang terintegrasi:** Deployment manual seringkali tidak memiliki mekanisme *rollback* otomatis; operator harus mengingat konfigurasi sebelumnya atau mengandalkan *backup* yang mungkin tidak konsisten [Rahman et al., 2023].
4. ***Absence of automatic drift detection*:** Tanpa *checking* berkala antara *state* aktual dan *desired state*, drift dapat terakumulasi selama periode lama tanpa terdeteksi [Zhang et al., 2026].

2.2 Continuous Deployment dan Paradigma GitOps

Continuous Deployment (CD) adalah praktik dalam ranah DevOps di mana setiap perubahan kode yang lulus pengujian secara otomatis di-deploy ke lingkungan produksi tanpa intervensi manual eksplisit [Limoncelli et al., 2018]. CD merupakan perluasan dari *Continuous Integration* (CI) dan bertujuan meminimalkan waktu antara penulisan kode dan ketersediaannya bagi pengguna akhir. Dalam ekosistem Kubernetes, CD memerlukan mekanisme yang dapat secara andal menyelaraskan perubahan konfigurasi dengan *state* klaster.

GitOps adalah paradigma spesifik di dalam CD yang menggunakan repositori Git sebagai *single source of truth* untuk mendefinisikan *state* infrastruktur dan aplikasi. CNCF GitOps Working Group mendefinisikan empat prinsip OpenGitOps: (1) sistem dideklarasikan secara deklaratif; (2) *state* sistem yang diinginkan tersimpan dan berversi dalam Git; (3) perubahan *state* ditarik (*pulled*) secara otomatis oleh agen

software; dan (4) agen *software* secara kontinu mendeteksi dan mengoreksi deviasi dari *state* yang diinginkan [CNCF GitOps Working Group, 2021].

Karakteristik GitOps yang berbeda dari pendekatan imperatif tradisional meliputi: *audit trail* lengkap melalui Git history, penerapan *code review* pada perubahan infrastruktur, *rollback* yang sederhana melalui `git revert`, dan reproduktibilitas konfigurasi di berbagai lingkungan [Yuen et al., 2021]. Dalam model *pull-based* GitOps, agen yang berjalan di dalam kluster secara periodik membandingkan *state* Git dengan *state* kluster dan menyelaraskan keduanya, menghilangkan kebutuhan akan akses kredensial kluster dari luar [Utami and Fatta, 2021].

ArgoCD mengimplementasikan prinsip-prinsip GitOps melalui *reconciliation loop* yang berjalan di dalam kluster. ArgoCD terdiri dari tiga komponen utama: (1) *application controller* yang secara periodik membandingkan *state* Git dengan *state* aktual dan melakukan tindakan korektif; (2) *repo server* yang mengelola koneksi ke repositori Git; dan (3) *Redis cache* yang menyimpan *state* sementara untuk performa [Argo Project, 2024]. ArgoCD berperan sebagai *negative feedback controller*: setiap deviasi (*drift*) bersifat *transien*, yaitu sistem secara otomatis kembali ke *desired state* tanpa memerlukan intervensi manusia, sepanjang *self-heal* diaktifkan.

2.3 Dasar Teori: *Reconciliation* dan *Convergence* pada Sistem Deklaratif

Karakteristik GitOps yang berbeda dari pendekatan imperatif dapat dijelaskan melalui dua prinsip teoretis yang saling berkaitan.

2.3.1 *Convergence Menuju Desired State*

Dalam teori sistem, konsep *negative feedback controller* menjelaskan bagaimana sistem yang menyimpan *desired state* dan secara kontinu berupaya mencapainya dikatakan bersifat *convergent* [Limoncelli et al., 2018]. ArgoCD mengimplementasikan prinsip ini melalui *reconciliation loop*: secara periodik (1) membaca *desired state* dari repositori Git, (2) membandingkannya dengan *state* aktual di kluster, dan (3) melakukan tindakan korektif jika terdapat deviasi [Argo Project, 2024]. Mekanisme ini menjamin bahwa setiap deviasi (*drift*) bersifat *transien*, yakni sistem secara otomatis kembali ke *desired state* tanpa memerlukan intervensi manusia, sepanjang *self-heal* diaktifkan.

Sebaliknya, pendekatan manual/imperatif tidak memiliki *feedback loop* bawaan. Setiap deviasi bersifat *persisten* sampai operator secara manual mendeteksi dan

mengoreksinya. Tanpa mekanisme otomatis, interval antara drift dan pemulihan bergantung sepenuhnya pada kewaspadaan dan ketersediaan operator, yang bervariasi secara tidak deterministik [Pohjola, 2025].

2.3.2 *Audit Trail sebagai Social Control Mechanism*

GitOps memanfaatkan Git sebagai *append-only log* yang merekam setiap perubahan konfigurasi beserta metadata (penulis, waktu, pesan commit). Prinsip keempat OpenGitOps, yaitu *continuous reconciliation*, bergantung pada kemampuan Git untuk merekonstruksi keputusan konfigurasi secara retroaktif [CNCF GitOps Working Group, 2021]. Hal ini tidak hanya meningkatkan *traceability*, tetapi juga berfungsi sebagai *social control mechanism*: setiap perubahan tunduk pada *code review*, sehingga mengurangi kemungkinan perubahan yang tidak terverifikasi masuk ke produksi.

2.4 Evaluasi Empiris GitOps dan ArgoCD

Utami and Fatta [2021] melakukan analisis perbandingan model deployment deklaratif *pull-based* (GitOps dengan ArgoCD) versus *push-based* pada Kubernetes, dan menemukan bahwa model *pull-based* dilaporkan lebih andal karena agen berjalan di dalam kluster dan tidak memerlukan kredensial eksternal. Shrestha and Ali [2024] mengevaluasi GitOps versus Ansible untuk manajemen konfigurasi Kubernetes, khususnya pada aspek *drift detection* dan *remedy time*; namun, studi ini mengandalkan penilaian kualitatif untuk menilai tingkat keparahan drift alih-alih metrik kuantitatif yang dapat direplikasi. Nataraja [2025] melakukan analisis berpusat keamanan terhadap pendekatan deklaratif (GitOps/ArgoCD) dan imperatif (Jenkins/`kubectl`), menemukan bahwa pendekatan deklaratif menghasilkan *drift correction* yang lebih cepat, *compliance rate* yang lebih tinggi, dan paparan kerentanan yang lebih rendah; namun, studi ini tidak mengisolasi efek pendekatan deployment dari variabel pengganggu seperti ukuran kluster dan keahlian tim.

Paavola [2021] membandingkan ArgoCD dan FluxCD untuk *multi-application management* pada Kubernetes dan melaporkan bahwa ArgoCD lebih sesuai untuk skenario multi-application, berkat dukungan UI dan integrasi yang lebih baik. D’Amore [2021] mengimplementasikan ArgoCD untuk *continuous deployment* pada lingkungan *hybrid cloud*, namun tidak membahas evaluasi metrik kuantitatif secara eksplisit. Matubber [2025] mengukur *reconciliation time*, *scalability*, dan *reli-*

ability GitOps untuk infrastruktur K8s, termasuk *self-healing* terhadap *configuration drift*; namun, pengukuran dilakukan tanpa *control group* yang menjalankan *manual workflow*, sehingga tidak dapat disimpulkan apakah *reconciliation time* yang terukur berbeda secara bermakna dari *manual recovery*.

Tabel 2.1 merangkum kelemahan metodologis kunci dari penelitian-penelitian di atas.

Tabel 2.1: Perbandingan Kelemahan Metodologis Penelitian Terdahulu

Penelitian	Fokus	Kelemahan Metodologis
Shrestha and Ali [2024]	GitOps vs Ansible	Penilaian kualitatif untuk tingkat keparahan drift; tidak ada analisis statistik formal
Nataraja [2025]	Keamanan deklaratif vs imperatif	Tidak mengisolasi variabel pengganggu; desain observasional
Matubber [2025]	Reconciliation time GitOps	Tanpa <i>control group</i> (<i>manual workflow</i>); tidak membandingkan secara terkontrol
Paavola [2021]	ArgoCD vs FluxCD	Komparatif deskriptif; tanpa metrik kuantitatif formal
D'Amore [2021]	ArgoCD hybrid cloud	Studi implementasi; tanpa evaluasi metrik

2.5 Metode Eksperimental Terkontrol pada Riset Sistem

Evaluasi empiris pada riset sistem dan infrastruktur memerlukan desain eksperimental yang memitigasi ancaman terhadap validitas [Wohlin et al., 2012]. Limoncelli et al. [2018] menjelaskan bahwa penelitian sistem harus membangun lingkungan yang dapat direproduksi, mengendalikan variabel pengganggu, dan mendefinisikan metrik yang jelas sebelum menyimpulkan hubungan kausal.

Desain eksperimental yang digunakan dalam penelitian ini membandingkan dua lingkungan yang menerima perlakuan identik (skenario drift yang sama) dalam kondisi terkontrol: Lingkungan A (deployment manual dengan `kubectl`) dan Lingkungan B (deployment GitOps dengan ArgoCD). Kedua lingkungan dibangun pada kluster identik dengan konfigurasi yang disamakan; satu-satunya perbedaan adalah paradigma deployment (variabel independen), sehingga perbedaan hasil pengukuran dapat diatribusikan kepada perbedaan paradigma [Wohlin et al., 2012].

Analisis data menggunakan uji *Mann–Whitney U* untuk membandingkan setiap metrik antara kedua lingkungan, disertai perhitungan ukuran efek (Cohen’s d atau Cliff’s δ) untuk mengkarakterisasi besarnya perbedaan yang teramati [Wohlin et al., 2012]. Prinsip-prinsip rigor yang diterapkan meliputi: (1) pengulangan (*replication*) untuk memastikan reliabilitas; (2) kontrol variabel pengganggu untuk menjaga validitas internal; (3) ukuran efek (*effect size*) untuk menilai signifikansi praktis; dan (4) analisis *power* untuk memastikan ukuran sampel memadai.

2.6 InvenioRDM sebagai Beban Uji (Workload)

InvenioRDM adalah platform repositori data penelitian *open-source* yang dikembangkan oleh CERN dan komunitas internasional, dirancang untuk memenuhi prinsip FAIR (*Findable, Accessible, Interoperable, Reusable*) [CERN, 2024]. Arsitektur InvenioRDM bersifat *microservices* yang terdiri dari beberapa komponen: (1) *web application* berbasis Flask/Gunicorn, (2) *Celery workers* untuk *asynchronous processing*, (3) PostgreSQL untuk *data storage*, (4) OpenSearch untuk *indexing* dan *search*, (5) Redis sebagai *cache* dan *message broker*, serta (6) MinIO/S3 untuk *object storage* [Przerwa and Rohr, 2025].

InvenioRDM dipilih sebagai beban uji karena memenuhi tiga kriteria kerja: (a) kompleksitas dependensi yang memerlukan urutan deployment, di mana *web* bergantung pada *database*, *cache*, dan *search engine*; (b) keberadaan komponen infrastruktur yang luas, mencakup *ingress*, TLS, *secrets*, monitoring, *backup*, dan kebijakan keamanan; dan (c) ketersediaan sebagai platform *open-source* yang dapat direproduksi [Chelakis, 2022, Fernández et al., 2016]. Dalam konteks penelitian ini, InvenioRDM berfungsi sebagai subjek uji (*test subject*), bukan objek penelitian. Segala temuan dan metrik berlaku untuk workload secara umum, tidak terbatas pada InvenioRDM.

2.7 Kesenjangan Penelitian (*Research Gap*)

Berdasarkan tinjauan pustaka di atas, terdapat empat kesenjangan yang saling berkaitan dan menjadi landasan penelitian ini:

1. **Kesenjangan Validitas Konstruk:** Penelitian sebelumnya menunjukkan bahwa GitOps mengurangi drift, namun temuan-temuan ini bergantung pada metrik yang tidak teroperasionalisasi secara kuantitatif. Shrestha and Ali [2024]

menggunakan penilaian kualitatif untuk tingkat keparahan drift, sementara Nataraja [2025] mengukur *compliance rate* tanpa mendefinisikan protokol pengukuran yang dapat direplikasi. Tanpa operasionalisasi metrik yang tegas, klaim karakteristik GitOps tidak dapat diverifikasi secara independen.

2. **Kesenjangan Validitas Internal:** Studi-studi yang ada tidak mengisolasi efek paradigma deployment dari variabel pengganggu. Nataraja [2025] tidak mengendalikan ukuran klaster atau keahlian operator; Matubber [2025] mengukur *reconciliation time* GitOps tanpa *control group* (*manual workflow*). Akibatnya, efek yang diamati tidak dapat diatribusikan secara kausal kepada GitOps, yang mungkin disebabkan oleh keahlian operator yang lebih tinggi, karakteristik *workload* yang lebih sederhana, atau kondisi klaster yang lebih stabil.
3. **Kesenjangan Metodologis:** Tidak ditemukan penelitian yang menerapkan desain eksperimental terkontrol dengan analisis statistik formal (uji komparatif dan ukuran efek) pada perbandingan GitOps versus manual. Absensi rigor statistik ini mengakibatkan temuan yang ada tidak dapat digeneralisasi dan rentan terhadap *Type I error* (menerima klaim positif yang sebenarnya palsu).
4. **Kesenjangan Kontekstual:** Evaluasi GitOps yang ada berfokus pada lingkungan berskala besar atau *cloud provider* komersial. Belum ada evaluasi pada konteks klaster berukuran kecil-sedang yang mengelola *workload* akademis seperti InvenioRDM, di mana sumber daya operasional terbatas dan satu insiden drift dapat mengancam integritas data penelitian.

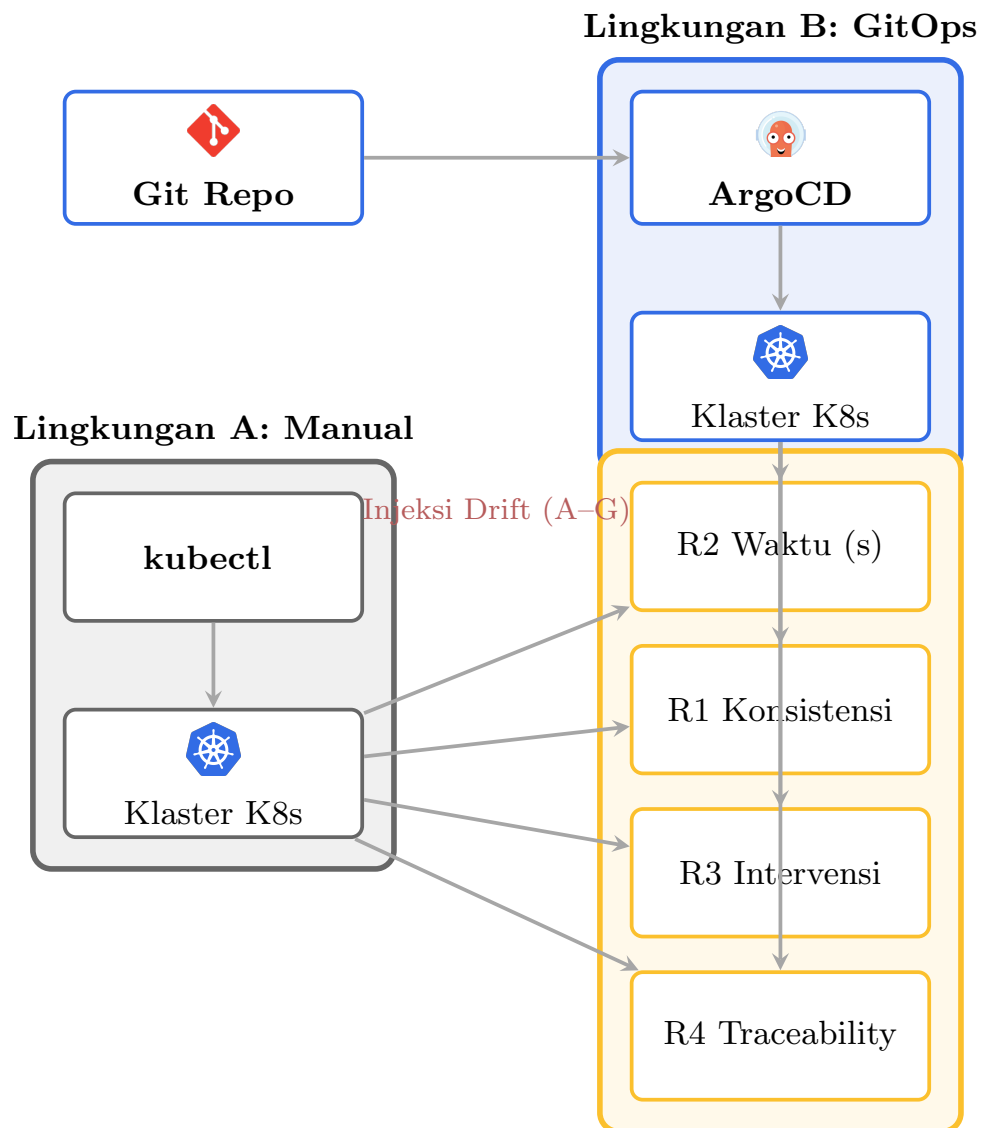
Penelitian ini mengisi keempat kesenjangan tersebut dengan: (1) mengoperasionalisasi tiga metrik kuantitatif yang dapat direplikasi; (2) mengendalikan variabel pengganggu melalui desain eksperimental terkontrol; (3) menerapkan analisis statistik komparatif dengan uji signifikansi dan ukuran efek; dan (4) mengevaluasi pada konteks klaster akademis dengan *workload* InvenioRDM.

BAB III

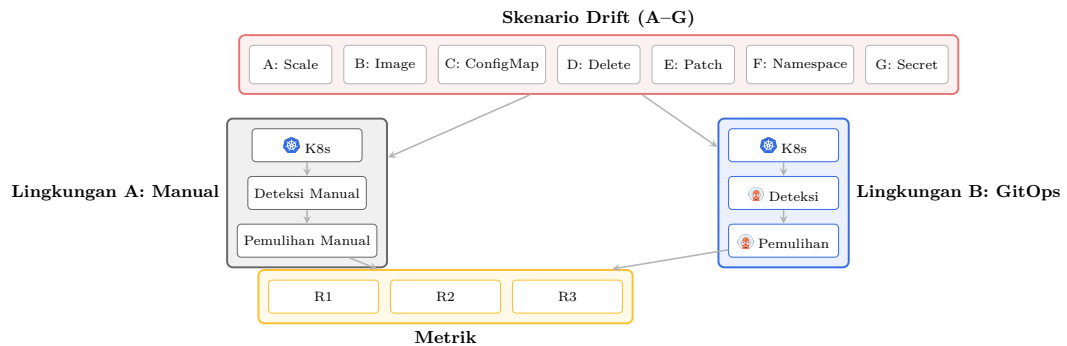
METODE DAN RANCANGAN PENELITIAN

3.1 Deskripsi Umum Penelitian

Penelitian ini mengevaluasi secara empiris dampak skenario *configuration drift* terhadap karakteristik operasional deployment pada dua paradigma manajemen yang berbeda, yaitu manual dan GitOps, pada sebuah kluster Kubernetes. Evaluasi dilakukan dengan membandingkan dua lingkungan paralel: Lingkungan A (deployment manual menggunakan `kubectl`) dan Lingkungan B (deployment GitOps menggunakan ArgoCD). Pada kedua lingkungan, *configuration drift* di-inject secara terkontrol melalui tujuh skenario yang dirancang untuk merepresentasikan kesalahan operasional yang umum terjadi. Hasil pengukuran terhadap tiga metrik utama (konsistensi deployment, *detection time/recovery time/root cause identification time*, dan *traceability*) dianalisis menggunakan uji statistik komparatif untuk mengkarakterisasi perbedaan antara kedua paradigma.



Gambar 3.1: Arsitektur eksperimen: perbandingan Lingkungan A (deployment manual dengan `kubectl`) dan Lingkungan B (deployment GitOps dengan ArgoCD). *Drift injection* dilakukan pada kedua lingkungan, dan metrik R1–R3 dikumpulkan dari masing-masing lingkungan.



Gambar 3.2: Skenario drift terkontrol (A–G): alur *injection* pada kedua lingkungan, mekanisme *detection* dan *recovery* pada Lingkungan A (manual) dan Lingkungan B (GitOps/ArgoCD), serta tiga metrik yang dikumpulkan.

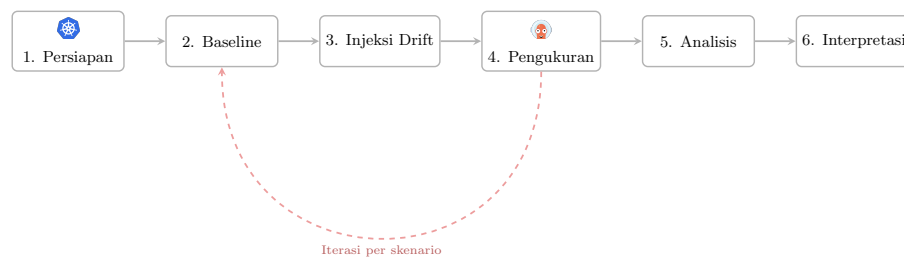
3.2 Metodologi

Metode penelitian yang digunakan adalah **Evaluasi Empiris Terkontrol**. Dalam desain ini, dua lingkungan paralel dibangun pada kluster Kubernetes yang identik: Lingkungan A (deployment manual) dan Lingkungan B (deployment GitOps). Kedua lingkungan menerima perlakuan identik berupa *drift injection* melalui skenario yang sama, sehingga satu-satunya variabel yang berbeda adalah paradigma deployment. Perbedaan hasil pengukuran antara kedua lingkungan dapat diatribusikan kepada perbedaan paradigma deployment. Urutan skenario diacak untuk menghindari efek urutan (*order effect*), dan variabel pengganggu dikendalikan melalui konfigurasi kluster yang disamakan [Wohlin et al., 2012].

Tahapan eksperimen dalam penelitian ini mengikuti rancangan *pre-test – treatment – post-test*:

1. **Tahap 1: Persiapan Lingkungan:** Pembangunan Lingkungan A (manual) dan Lingkungan B (GitOps) pada kluster Kubernetes yang identik. Konfigurasi kluster, versi ArgoCD, dan spesifikasi node disamakan untuk mengendalikan variabel pengganggu.
2. **Tahap 2: Kalibrasi Baseline (O1):** Pengukuran *baseline* pada keadaan bersih sebelum *drift injection*. Mencakup verifikasi status deployment, pengukuran awal konsistensi, dan *logging* metrik lingkungan (penggunaan CPU/memori, latensi jaringan).

3. **Tahap 3: Drift Injection (X):** *Controlled configuration drift injection* pada kedua lingkungan menggunakan perintah identik (mis. `kubectl edit`, `kubectl scale`, `kubectl patch`). Urutan skenario diacak untuk menghindari efek urutan (*order effect*).
4. **Tahap 4: Pengukuran Pasca-Perlakuan (O2):** Pengumpulan data metrik pasca-*injection*, termasuk *detection time*, *recovery time*, *root cause identification time*, dan dokumentasi *traceability*.
5. **Tahap 5: Analisis Statistik Komparatif:** Penerapan uji *Mann–Whitney U* serta perhitungan ukuran efek (Cohen’s d atau Cliff’s δ) untuk mengkarakterisasi perbedaan antara kedua paradigma.
6. **Tahap 6: Interpretasi dan Kesimpulan:** Pembahasan hasil statistik dalam konteks literatur dan praktik operasional.



Gambar 3.3: Bagan alir eksperimen: tahapan penelitian dari persiapan hingga interpretasi. Iterasi per skenario menunjukkan pengulangan acak dari baseline hingga pengukuran untuk setiap skenario drift.

3.3 Definisi Intervensi dan Variabel Penelitian

3.3.1 Intervensi (Perlakuan)

Intervensi dalam penelitian ini adalah **drift injection controlled** melalui tujuh skenario (A–G). Intervensi ini bersifat *identik pada kedua lingkungan*, yakni perintah `kubectl` yang sama digunakan untuk *inject* drift pada Lingkungan A dan Lingkungan B. Yang membedakan kedua lingkungan bukanlah perlakuannya, melainkan paradigma manajemen deployment yang menentukan bagaimana sistem merespons terhadap drift yang terjadi.

3.3.2 Variabel Independen (Variabel Bebas)

Variabel independen adalah **paradigma manajemen deployment** dengan dua level:

1. **Lingkungan A, Deployment Manual:** Deployment dan modifikasi dilakukan langsung pada klaster menggunakan `kubectl apply`, `kubectl edit`, `kubectl scale`, dan perintah imperatif lainnya tanpa repositori Git sebagai *source of truth*.
2. **Lingkungan B, Deployment GitOps (ArgoCD):** Deployment dikelola secara deklaratif melalui repositori Git; ArgoCD berjalan di dalam klaster sebagai agen *reconciliation* yang secara kontinu menyelaraskan *state* klaster dengan *state* di Git.

3.3.3 Variabel Dependen (Variabel Terikat)

Tiga metrik utama diukur sebagai variabel dependen:

Metrik R1: Konsistensi Deployment Definisi: Persentase sumber daya klaster (*Pod*, *Deployment*, *ConfigMap*, *Service*) yang cocok dengan *desired declarative state* setelah skenario drift.

Cara Ukur:

$$\text{Konsistensi (\%)} = \frac{\text{Jumlah sumber daya dengan status Healthy/Synced}}{\text{Total sumber daya yang diharapkan}} \times 100$$

Untuk Lingkungan B, status diambil dari ArgoCD Application status. Untuk Lingkungan A, status diverifikasi menggunakan `kubectl get` dan `kubectl describe`. Konsistensi diukur pada t_0 (sebelum drift), t_1 (sebelum *recovery*), dan t_2 (setelah *recovery*).

Metrik R2: Waktu Deteksi, Pemulihan, dan Identifikasi Akar Masalah Definisi: Tiga sub-metrik waktu (dalam detik) yang mengukur kecepatan respons terhadap *configuration drift*.

- **Detection Time ($t_d - t_0$):** Waktu dari *drift injection* hingga drift terdeteksi. Untuk Lingkungan B, ArgoCD menunjukkan status *OutOfSync* pada aplikasi

yang terkena drift (dipantau menggunakan `argocd app get` dan log ArgoCD). Untuk Lingkungan A, operator mendeteksi deviasi secara manual melalui `kubectl get` atau *monitoring*.

- **Recovery Time** ($t_2 - t_0$): Waktu dari *drift injection* hingga *state* klaster kembali ke *baseline*. Untuk Lingkungan B, ArgoCD menyelesaikan *automatic reconciliation* (*self-heal*) dan menunjukkan status *Synced/Healthy*. Untuk Lingkungan A, operator menyelesaikan seluruh perintah `kubectl` manual yang diperlukan untuk mengembalikan *state* ke *baseline*.
- **Root Cause Identification Time** ($t_{rci} - t_d$): Waktu dari deteksi drift hingga penyebab drift teridentifikasi. Untuk Lingkungan B, ArgoCD menampilkan *diff* antara *state* Git dan *state* klaster, sehingga akar masalah dapat langsung diidentifikasi. Untuk Lingkungan A, operator harus memeriksa log, menjalankan `kubectl describe`, dan menganalisis perubahan secara manual untuk menentukan penyebab drift.

Pengamatan Tambahan: Jumlah perintah `kubectl` yang diperlukan untuk pemulihan pada Lingkungan A dicatat sebagai pengamatan pelengkap, bukan sebagai metrik independen, karena waktu pemulihan (*recovery time*) secara inheren sudah mencerminkan kompleksitas pemulihan.

Pengulangan: Setiap skenario diulang $n = 10$ kali per lingkungan untuk menghasilkan distribusi waktu yang memadai untuk analisis statistik.

Metrik R3: Traceability Operasional (Deskriptif) **Definisi:** Kemampuan untuk merekonstruksi siapa yang mengubah *state*, apa yang berubah, kapan perubahan terjadi, dan bagaimana *recovery* dilakukan.

Cara Ukur: Metrik ini dinilai secara deskriptif, bukan diuji secara statistik. Untuk setiap skenario drift, aspek *traceability* berikut didokumentasikan untuk kedua lingkungan:

- **Lingkungan B (GitOps):** Setiap perubahan direkam secara otomatis melalui Git *commit history* (penulis, *timestamp*, *diff*) dan ArgoCD *notifications*. Kelengkapan *audit log* dihitung sebagai persentase perubahan yang dapat direkonstruksi dari Git *commit history*.

- **Lingkungan A (Manual):** *Traceability* bergantung pada Kubernetes *audit logs* dan *manual logging*. Kelengkapan *audit log* dihitung sebagai persentase perubahan yang dapat direkonstruksi dari *audit logs*.

Penjelasan Metodologis: Metrik *traceability* ini tidak diuji secara statistik karena perbedaan struktural antara kedua paradigma sudah terjamin oleh desain sistem: Git secara inheren menyediakan *audit trail* lengkap (*commit hash, author, timestamp, diff*), sementara *manual deployment* bergantung pada *audit logs* yang parsial. Perbandingan kuantitatif pada dimensi ini bersifat *predetermined*. Sebagai gantinya, *traceability* dideskripsikan secara kualitatif dalam pembahasan hasil, mencakup: kelengkapan jejak perubahan, kemudahan identifikasi akar masalah, dan keterkaitan dengan *root cause identification time* (R2).

3.4 Peran Peneliti dan Standarisasi Prosedur

Dalam penelitian ini, peneliti berperan sebagai **instrumen pengukuran yang terstandarisasi**, bukan sebagai variabel subjek. Hal ini penting untuk menjaga validitas internal mengingat peneliti juga bertindak sebagai operator pada Lingkungan A. Peran dan batasan peneliti didefinisikan sebagai berikut:

1. **Pada Lingkungan A (Manual):** Peneliti mengeksekusi prosedur *recovery* berdasarkan *recovery runbook* yang telah ditentukan sebelumnya (*pre-defined*). Operator tidak membuat keputusan subjektif selama *recovery*, sehingga setiap langkah korektif mengikuti skrip yang telah ditulis. Seluruh *timestamp* (awal dan akhir setiap perintah) direkam secara otomatis oleh skrip otomatisasi, bukan oleh *manual logging*.
2. **Pada Lingkungan B (GitOps):** ArgoCD beroperasi secara otomatis; peneliti hanya memantau dan merekam data. Tidak ada keputusan operasional yang dibuat oleh peneliti pada lingkungan ini.
3. **Pencatatan data:** Seluruh pengukuran waktu (R2) dan konsistensi (R1) direkam oleh skrip otomatisasi, bukan oleh penilaian manusia. *Traceability* (R3) didokumentasikan berdasarkan bukti objektif (log dan Git history) tanpa penilaian subjektif.

Batasan penggunaan operator tunggal diakui secara eksplisit sebagai ancaman terhadap validitas eksternal (dibahas pada § 3.13) dan direkomendasikan untuk replikasi dengan tim operator yang berbeda.

3.5 Skrip Otomasi Eksperimen

Untuk memastikan konsistensi prosedural dan mengurangi bias eksperimenter, empat kategori skrip otomasi dikembangkan:

3.5.1 Skrip *Drift Injection* (`drift-inject.sh`)

Skrip ini mengotomasi *injection* skenario drift pada kedua lingkungan:

- Memilih skenario secara acak dari himpunan {A, B, C, D, E, F, G}.
- Mengeksekusi perintah `kubectl` yang sesuai pada kedua lingkungan secara paralel.
- Merekam *timestamp* t_0 (waktu *injection*) ke log terstruktur (JSON/CSV).
- Memverifikasi bahwa drift berhasil diterapkan sebelum melanjutkan ke tahap pengukuran.

3.5.2 Skrip Pengumpulan Metrik (`metrics-collect.sh`)

Skrip ini melakukan polling berkala terhadap kedua lingkungan:

- **Lingkungan B:** Memanggil ArgoCD API (`argocd app get`) setiap 5 detik untuk mendeteksi perubahan status (*Synced* $\rightarrow$ *OutOfSync* $\rightarrow$ *Synced*).
- **Lingkungan A:** Menjalankan `kubectl get` dan `kubectl describe` secara berkala untuk memantau status sumber daya.
- Merekam semua *timestamp* secara otomatis (*detection time* t_d , *recovery time* t_2).
- Menghitung skor konsistensi (R1) secara otomatis berdasarkan perbandingan *state* aktual dengan *desired state*.

3.5.3 Skrip Verifikasi Baseline (**baseline-check.sh**)

Skrip ini memverifikasi kondisi klaster sebelum setiap *run* eksperimental:

- Memeriksa bahwa semua *pod* dalam status *Running* dan *healthy*.
- Memastikan penggunaan CPU/memori node di bawah 70% kapasitas.
- Memverifikasi bahwa ArgoCD menunjukkan status *Synced/Healthy* pada Lingkungan B.
- Menolak memulai eksperimen jika syarat baseline tidak terpenuhi; mencatat pelanggaran ke log.

3.5.4 Skrip Recovery Runbook (**recovery-runbook.sh**)

Skrip ini memandu operator pada Lingkungan A melalui prosedur *recovery* yang terstandarisasi:

- Menampilkan langkah-langkah korektif secara berurutan sesuai jenis drift yang terjadi.
- Merekam *timestamp* awal dan akhir untuk setiap perintah `kubectl` yang dieksekusi.
- Menghitung jumlah perintah `kubectl` yang dieksekusi sebagai pengamatan pelengkap.
- Memastikan operator tidak melompati langkah atau membuat keputusan di luar prosedur.

Seluruh skrip akan dipublikasikan sebagai *artifact* pendamping untuk mendukung reproduktibilitas penelitian.

3.6 Lingkungan Eksperimen

3.6.1 Klaster Kubernetes

Penelitian menggunakan klaster Kubernetes yang dikelola Rancher dengan spesifikasi hardware sebagai berikut:

- **Jumlah Node:** 3 node

- **Total CPU:** 24 inti
- **Total Memori:** ~23 Gi
- **Versi Kubernetes:** v1.29+
- **Container Runtime:** containerd
- **Sistem Operasi Node:** Ubuntu 22.04 LTS

3.6.2 ArgoCD (Lingkungan B)

- **Versi ArgoCD:** v2.12+
- **Konfigurasi Sync:** *Auto-sync* diaktifkan. Interval *sync* ditetapkan pada 60 detik untuk konsistensi *detection time*.
- **Konfigurasi Self-Heal:** Diaktifkan untuk seluruh eksperimen, sehingga ArgoCD secara otomatis mendeteksi dan mengoreksi drift.
- **Konfigurasi Prune:** *Pruning* diaktifkan agar sumber daya yang dihapus secara manual (Skenario D) dapat dideteksi dan diciptakan kembali oleh ArgoCD.

3.6.3 Beban Uji (Workload)

Deployment InvenioRDM dengan 16+ komponen: *web*, *worker*, PostgreSQL, OpenSearch, Redis, MinIO, Traefik, Cert-manager, Sealed Secrets, monitoring, *backup*, dan kebijakan keamanan. InvenioRDM merupakan subjek uji, bukan objek penelitian.

3.6.4 Kontrol Variabel Pengganggu

Agar validitas internal tetap tinggi, variabel berikut dikendalikan secara sistematis:

1. **Interval Sync ArgoCD:** Ditetapkan pada 60 detik (bukan default 180 detik) agar *detection time* konsisten.
2. **Tekanan Sumber Daya Node:** Rekam penggunaan CPU/memori kluster sebelum setiap pengujian. Hanya jalankan eksperimen jika node berada di bawah 70% kapasitas.

3. **Latensi Jaringan:** Gunakan klaster *on-premise* dengan latensi stabil. Catat RTT antar-node.
4. **Drift Injection Time:** *Injection* selalu dilakukan pada interval *sync* ArgoCD untuk menghindari *edge case* waktu tunggu yang tidak dapat diprediksi.
5. **Versi ArgoCD:** Gunakan satu versi ArgoCD (v2.12+) sepanjang eksperimen.

3.6.5 Protokol Monitoring Variabel Terkontrol

Variabel pengganggu dipantau secara kontinu selama eksperimen menggunakan `baseline-check.sh` yang mencatat metrik sistem setiap 10 detik. Jika selama eksperimen terjadi pelanggaran batas (misalnya penggunaan CPU melebihi 70%), eksperimen dihentikan, data dari *run* tersebut dibuang, lingkungan dikalibrasi ulang ke baseline, dan *run* diulang. Seluruh kejadian pelanggaran dicatat dalam log eksperimen untuk transparansi.

3.7 Analisis Power dan Justifikasi Ukuran Sampel

Ukuran sampel ditentukan berdasarkan analisis *power a priori* menggunakan pendekatan berikut:

1. **Ukuran efek target:** Berdasarkan temuan Nataraja [2025] yang melaporkan perbedaan substansial antara pendekatan deklaratif dan imperatif, ukuran efek besar ($d = 1,0$, Cohen's convention) digunakan sebagai estimasi konservatif.
2. **Power dan signifikansi:** *Statistical power* ditetapkan pada $1 - \beta = 0,80$ dan $\alpha = 0,05$ (two-tailed).
3. **Ukuran sampel minimum (per grup):** Berdasarkan Cohen's convention untuk ukuran efek besar ($d = 1,0$), $\alpha = 0,05$, dan $power = 0,80$, ukuran sampel minimum per grup adalah $n = 17$ secara total per metrik [Cohen, 1988]. Namun, karena setiap skenario diulang secara independen (7 skenario $\times n$ pengulangan per skenario), jumlah total observasi per metrik per lingkungan adalah $7 \times n$.
4. **Pemilihan n per skenario:** Untuk R2 (*detection time*, *recovery time*, dan *root cause identification time*, metrik utama), dipilih $n = 10$ per skenario per

lingkungan, menghasilkan $7 \times 10 = 70$ observasi per lingkungan, jauh melampaui minimum $n = 17$ secara total. Untuk R1 (konsistensi), dipilih $n = 5$ per skenario per lingkungan, menghasilkan $7 \times 5 = 35$ observasi per lingkungan, masih memadai untuk mendeteksi efek besar. R3 (*traceability*) bersifat deskriptif dan tidak memerlukan ukuran sampel statistik. Jika hasil menunjukkan efek sedang ($d = 0,5$), pengulangan dapat ditambah pada tahap pengumpulan data tambahan.

3.8 Matriks Eksperimen

Tabel 3.1 menunjukkan rancangan eksperimental secara keseluruhan.

Tabel 3.1: Matriks Eksperimen

Dimensi	Lingkungan A (Manual)	Lingkungan B (GitOps)
Paradigma	kubectl imperatif	ArgoCD deklaratif + Git
Intervensi	Skenario drift A–G (identik)	Skenario drift A–G (identik)
Pengulangan	$n = 5$ per skenario (R1); $n = 10$ per skenario (R2); R3 deskriptif	$n = 5$ per skenario (R1); $n = 10$ per skenario (R2); R3 deskriptif
Metrik	R1, R2, R3	R1, R2, R3
Baseline	kubectl get semua sumber daya sebelum drift	ArgoCD App status sebelum drift

3.9 Skenario Drift Terkontrol

Tujuh skenario drift dirancang untuk merepresentasikan kesalahan operasional yang umum. Setiap skenario dieksekusi secara identik pada kedua lingkungan. Untuk Lingkungan A, operator kemudian memulihkan secara manual mengikuti *recovery runbook*. Untuk Lingkungan B, ArgoCD secara otomatis mendeteksi dan memulihkan drift (*self-heal* aktif).

3.9.1 Skenario A: Perubahan Replika (Replica Drift)

Secara manual mengubah jumlah replika Deployment menggunakan `kubectl scale deployment/nama-deployment --replicas=5` ketika *desired* adalah 3. Perubahan ini melewati Git pada Lingkungan B.

3.9.2 Skenario B: Perubahan Image (Image Drift)

Secara manual mengganti versi image kontainer pada Deployment menggunakan `kubectl edit deployment/nama-deployment` dan mengubah kolom image ke versi yang berbeda.

3.9.3 Skenario C: Perubahan ConfigMap (ConfigMap Drift)

Mengubah isi ConfigMap langsung di dalam kluster menggunakan `kubectl edit configmap/nama-configmap`. Memodifikasi parameter konfigurasi seperti `DATABASE_URL` atau `LOG_LEVEL`.

3.9.4 Skenario D: *Resource Deletion*

Menghapus Deployment atau Service secara manual menggunakan `kubectl delete deployment/nama-deployment`. Untuk Lingkungan B, ArgoCD dengan *prune* aktif akan mendeteksi sumber daya yang hilang dan menciptakan ulang dari manifest Git.

3.9.5 Skenario E: Patch Manual Tidak Sah (Unauthorized Manual Patch)

Menerapkan `kubectl patch` langsung terhadap sumber daya yang dikelola, mis. mengubah *resource limits* pada sebuah Pod menggunakan patch JSON.

3.9.6 Skenario F: Drift Lintas Namespace (Cross-namespace Drift)

Menerapkan sumber daya ke namespace yang salah secara manual. Mengukur apakah GitOps membatasi propagasi atau mendeteksi kesalahan namespace secara otomatis. Contoh: `kubectl apply -f deployment.yaml --namespace=wrong-namespace`.

3.9.7 Skenario G: Perubahan Secret (Secret Drift)

Mengubah secret secara manual di luar Git. Skenario ini sangat umum dalam praktik produksi. Contoh perintah:

Listing III.1: Contoh perubahan Secret

```
1 kubectl patch secret my-secret \
2   --patch='{"data":{"password":"<new_base64>"}}'
```

3.10 Protokol Pengukuran

3.10.1 Protokol Pengukuran Baseline (O1)

Sebelum setiap *run*:

1. Jalankan `baseline-check.sh` untuk memverifikasi kondisi klaster.
2. Verifikasi semua *pod* dalam status *Running* dan *healthy*.
3. Catat *timestamp* awal.
4. Simpan output `kubectl get all -n <namespace>` (Lingkungan A) atau `argocd app list` (Lingkungan B) sebagai baseline.
5. Verifikasi variabel pengganggu berada dalam batas (CPU < 70%, RTT stabil).

3.10.2 Protokol *Drift Injection* (X)

1. Jalankan `drift-inject.sh` yang memilih skenario secara acak dari daftar {A, B, C, D, E, F, G}.
2. Skrip mengeksekusi perintah *drift injection* pada kedua lingkungan secara paralel.
3. Skrip mencatat *timestamp* t_0 (waktu eksekusi perintah *injection*) secara otomatis.

3.10.3 Protokol Pengukuran Pasca-*Injection* (O2)

1. **Untuk Lingkungan B (GitOps):** `metrics-collect.sh` memantau status ArgoCD setiap 5 detik. Catat: (a) waktu ArgoCD menunjukkan *OutOfSync* (*detection*); (b) waktu ArgoCD menunjukkan *Synced* setelah *auto-heal* (*recovery*); (c) waktu yang diperlukan untuk mengidentifikasi akar masalah dari *diff* yang ditampilkan ArgoCD (*root cause identification*). Hitung $t_d - t_0$ (*detection time*), $t_2 - t_0$ (*total recovery time*), dan $t_{rci} - t_d$ (*root cause identification time*).
2. **Untuk Lingkungan A (Manual):** Operator menjalankan `recovery-runbook.sh` yang memandu *recovery* berdasarkan jenis drift. `metrics-collect.sh` mencatat waktu eksekusi setiap perintah korektif. Catat: (a) waktu operator mendeteksi deviasi (*detection*); (b) waktu

operator menyelesaikan seluruh perintah korektif (*recovery*); (c) waktu operator mengidentifikasi penyebab drift melalui pemeriksaan log dan `kubectl describe` (*root cause identification*). Hitung $t_d - t_0$, $t_2 - t_0$, dan $t_{rci} - t_d$.

3. Catat jumlah perintah `kubectl` yang diperlukan sebagai pengamatan pelengkap, direkam otomatis oleh `recovery-runbook.sh`.
4. Dokumentasikan *traceability*: catat kelengkapan *audit log* untuk Lingkungan B (Git *commit history* + ArgoCD *notifications*) dan Lingkungan A (Kubernetes *audit logs*).

3.11 Metode Pengujian (Eksperimen E1 s.d. E3)

3.11.1 E1: Konsistensi Deployment (R1)

Tabel 3.2: Desain Eksperimen E1: Konsistensi Deployment

Parameter	Deskripsi
Metrik	Persentase sumber daya yang cocok dengan <i>desired declarative state</i>
Cara Ukur	Hitung jumlah sumber daya Healthy/Synced dibagi total <i>expected resources</i>
Paradigma	(a) Manual/ <code>kubectl</code> , (b) GitOps/ArgoCD
Pengulangan	n = 5 per skenario per lingkungan

3.11.2 E2: Waktu Deteksi, Pemulihan, dan Identifikasi Akar Masalah (R2)

Tabel 3.3: Desain Eksperimen E2: Waktu Deteksi, Pemulihan, dan Identifikasi Akar Masalah

Parameter	Deskripsi
Metrik	<i>Detection time</i> ($t_d - t_0$), <i>Recovery time</i> ($t_2 - t_0$), <i>Root cause identification time</i> ($t_{rci} - t_d$)
Cara Ukur	Lingkungan B: ArgoCD API <i>polling</i> setiap 5 detik; Lingkungan A: <code>kubectl commands</code> + <i>operator runbook</i>
Paradigma	(a) Manual/ <code>kubectl</code> , (b) GitOps/ArgoCD (<i>self-heal</i> aktif)
Pengulangan	n = 10 per skenario per lingkungan

3.11.3 E3: *Traceability* Operasional (R3, Deskriptif)

Tabel 3.4: Desain Eksperimen E3: *Traceability* Operasional (Deskriptif)

Parameter	Deskripsi
Metrik	Kelengkapan <i>audit log</i> (%), kemudahan identifikasi akar masalah (deskriptif)
Cara Ukur	Lingkungan B: persentase perubahan yang dapat direkonstruksi dari Git <i>commit history</i> ; Lingkungan A: persentase perubahan yang dapat direkonstruksi dari Kubernetes <i>audit logs</i>
Paradigma	(a) Manual/ <code>kubectl</code> , (b) GitOps/ArgoCD
Catatan	Metrik ini dinilai secara deskriptif, bukan diuji secara statistik

3.12 Analisis Statistik Komparatif

Analisis ini bersifat eksploratif-komparatif, bukan konfirmatoris. Tujuannya adalah mengkarakterisasi perbedaan antara kedua paradigma deployment dan mengukur besarnya efek yang teramati, bukan menguji hipotesis nol secara formal.

Setelah pengumpulan data selesai, analisis statistik dilakukan dengan langkah berikut:

1. **Statistik Deskriptif:** Hitung mean, median, rentang, dan deviasi standar untuk setiap metrik per lingkungan per skenario. Presentasikan dalam bentuk *box plot* per skenario dan per metrik.
2. **Uji Perbandingan:** Terapkan *Mann–Whitney U test* untuk membandingkan setiap metrik antara Lingkungan A dan Lingkungan B. Uji ini dipilih karena tidak mengasumsikan distribusi normal dan cocok untuk ukuran sampel yang digunakan dalam penelitian ini. Tingkat signifikansi ditetapkan pada $\alpha = 0,05$.
3. **Ukuran Efek:** Hitung Cohen's d sebagai ukuran efek standar yang menunjukkan besarnya perbedaan praktis antara kedua kelompok. Interpretasi mengikuti konvensi Cohen: $|d| < 0,2$ (efek kecil), $|d| \approx 0,5$ (efek sedang), $|d| > 0,8$ (efek besar).

Metrik R3 (*Traceability*) dinilai secara deskriptif dan tidak diuji secara statistik.

3.13 Ancaman terhadap Validitas

Berikut adalah identifikasi ancaman terhadap validitas beserta strategi mitigasi yang diterapkan, mengikuti kerangka Wohlin et al. [2012] dan Cook and Campbell [1979].

3.13.1 Validitas Internal

1. **Seleksi (*Selection Bias*):** Kedua lingkungan tidak ditugaskan secara acak dari populasi, melainkan peneliti membangun keduanya secara deliberatif. *Mitigasi:* Lingkungan A dan B dibangun pada kluster identik dengan konfigurasi yang disamakan; satu-satunya perbedaan adalah paradigma deployment (variabel independen).
2. **Operator Tunggal (*Single Operator Bias*):** Peneliti bertindak sebagai satu-satunya operator pada Lingkungan A, yang dapat membatasi generalisasi. *Mitigasi:* (a) Operator mengikuti *recovery runbook* yang terstandarisasi, sehingga keputusan subjektif diminimalkan; (b) seluruh *timestamp* direkam oleh skrip otomatis, bukan oleh manusia; (c) metrik yang tidak bergantung pada operator (R1 konsistensi, R2 deteksi otomatis ArgoCD) memberikan pengukuran objektif; (d) batasan ini diakui secara eksplisit dan direkomendasikan untuk replikasi dengan operator berbeda.
3. **Maturasi (*Maturation*):** Kinerja operator dapat membaik sepanjang eksperimen karena pembelajaran. *Mitigasi:* Urutan skenario diacak; data pembelajaran operator dicatat; *practice run* dilakukan sebelum pengumpulan data resmi.
4. **Instrumentasi:** Perubahan pada alat ukur selama eksperimen (mis. pembaruan ArgoCD). *Mitigasi:* Seluruh versi software (ArgoCD, Kubernetes, InvenioRDM) dikunci sebelum eksperimen dimulai dan tidak diperbarui selama pengumpulan data.
5. **Efek Pengujian (*Testing*):** Operator mungkin berubah perilakunya karena sadar sedang diobservasi (*Hawthorne effect*). *Mitigasi:* Operator mengikuti *recovery runbook* terstandarisasi; waktu reaksi direkam oleh skrip, bukan oleh penilaian manusia.

3.13.2 Validitas Eksternal

1. **Kekhususan Konteks:** Hasil mungkin tidak generalisasi ke klaster berukuran besar, *workload* yang berbeda, atau tim operator yang lebih besar. *Mitigasi:* Dokumentasikan seluruh karakteristik klaster, *workload*, dan operator secara transparan; gunakan InvenioRDM yang mewakili kelas *workload* multi-service umum.
2. **Interaksi Seleksi dan Perlakuan:** Efek paradigma deployment mungkin spesifik untuk tingkat keahlian operator dalam penelitian ini. *Mitigasi:* Catat profil keahlian operator; rekomendasikan replikasi dengan tim yang berbeda.
3. **Operator Tunggal:** Penggunaan satu operator membatasi generalisasi ke konteks tim yang beragam. *Mitigasi:* Standarisasi prosedur melalui runbook dan skrip otomasi; dokumentasikan batasan secara eksplisit; rekomendasikan replikasi multi-operator sebagai penelitian lanjutan.

3.13.3 Validitas Konstruk

1. **Bias Operasi Tunggal (*Mono-operation Bias*):** Menggunakan satu *workload* (InvenioRDM) dan satu alat GitOps (ArgoCD). *Mitigasi:* InvenioRDM dipilih karena kompleksitas yang mewakili kelas *workload* multi-service; ArgoCD merupakan implementasi GitOps yang paling banyak digunakan.
2. **Bias Metode Tunggal (*Mono-method Bias*):** Metrik R1 dan R2 diukur secara otomatis oleh skrip otomasi; R3 bersifat deskriptif dan didokumentasikan berdasarkan bukti objektif (log dan Git history). *Mitigasi:* Seluruh metrik kuantitatif menggunakan pengukuran objektif berbasis *timestamp*; R3 dideskripsikan berdasarkan data yang dapat diverifikasi secara independen.
3. **Operasionalisasi Konstruk:** Definisi “drift” dibatasi pada 7 skenario yang merepresentasikan kesalahan operasional umum. *Mitigasi:* Skenario dirancang berdasarkan taksonomi Zhang et al. [2026] dan praktik yang dilaporkan oleh Pohjola [2025]; batasan ini diakui secara eksplisit.

3.13.4 Reliabilitas

1. **Reliabilitas Pengukuran:** Pengukuran waktu (R2) bergantung pada presisi *logging timestamp*. *Mitigasi:* Semua *timestamp* direkam secara otomatis oleh

skrip otomasi; akurasi diverifikasi dengan Clock *synchronization* (NTP).

2. **Reliabilitas Replikasi:** Pihak lain harus dapat mereplikasi eksperimen. *Mitigasi:* Protokol eksperimen didokumentasikan secara rinci; konfigurasi klaster, versi software, dan seluruh skrip otomasi (`drift-inject.sh`, `metrics-collect.sh`, `baseline-check.sh`, `recovery-runbook.sh`) akan dipublikasikan sebagai *artifact* pendamping.

BAB IV

JADWAL PENELITIAN

Penelitian ini direncanakan untuk dilaksanakan selama delapan bulan, dari bulan Juni 2026 hingga Januari 2027. Jadwal penelitian disusun berdasarkan tahapan evaluasi empiris terkontrol: persiapan lingkungan, pengembangan skrip otomasi, kalibrasi baseline, pelaksanaan eksperimen, analisis statistik, dan penulisan laporan.

Tabel 4.1 menunjukkan rencana jadwal pelaksanaan penelitian berbasis bulan. Simbol • menunjukkan bahwa kegiatan dilaksanakan pada bulan tersebut.

Tabel 4.1: Jadwal Penelitian (Gantt Chart)

Kegiatan	Bulan							
	Jun	Jul	Agu	Sep	Okt	Nov	Des	Jan
Studi Literatur	•	•						
Perancangan Lingkungan		•	•					
Implementasi ArgoCD + Baseline Manual			•	•				
Pengembangan Skrip Otomasi		•	•	•				
Persiapan Baseline & Kalibrasi				•	•			
Eksperimen & Pengumpulan Data (E1–E3)					•	•		
Analisis Statistik & Interpretasi						•	•	
Penulisan Laporan	•	•	•	•	•	•	•	•
Seminar / Sidang								•

4.1 Penjelasan Kegiatan

1. Studi Literatur (Juni – Juli 2026)

Kegiatan ini mencakup pengumpulan dan analisis literatur mengenai: *configuration drift* pada Kubernetes, evaluasi empiris GitOps, metode eksperimental pada riset sistem, dan arsitektur InvenioRDM sebagai beban uji. Sumber literatur meliputi jurnal ilmiah (IEEE, ACM, Springer), dokumentasi resmi, buku teks, dan artikel teknis. Tujuan dari fase ini adalah membangun landasan teoretis yang kuat.

2. Perancangan Lingkungan (Juli – Agustus 2026)

Tahap ini meliputi perancangan arsitektur Lingkungan A (deployment manual/`kubectl`) dan Lingkungan B (deployment GitOps/ArgoCD) pada kluster

Kubernetes yang sama. Perancangan meliputi: definisi variabel dan intervensi, pemilihan tujuh skenario drift, spesifikasi protokol pengukuran untuk tiga metrik (R1, R2, R3), dan rancangan eksperimen.

3. Implementasi ArgoCD + Baseline Manual (Agustus – September 2026)

Tahap implementasi meliputi: konfigurasi ArgoCD pada klaster, pembuatan repositori GitOps dengan manifest deklaratif, konfigurasi baseline manual (*workflow* `kubectl`), serta konfigurasi *logging* dan *monitoring* (Prometheus, Grafana, Kubernetes audit logs, ArgoCD notifications). Pada akhir fase ini, kedua lingkungan siap untuk eksperimen.

4. Pengembangan Skrip Otomasi (Juli – September 2026)

Pengembangan empat skrip otomasi yang mendukung konsistensi prosedural eksperimen: (a) `drift-inject.sh` untuk *injection* skenario drift secara acak dengan *logging timestamp*; (b) `metrics-collect.sh` untuk pengumpulan metrik secara berkala via ArgoCD API dan `kubectl`; (c) `baseline-check.sh` untuk verifikasi kondisi klaster sebelum setiap *run*; dan (d) `recovery-runbook.sh` untuk memandu operator pada Lingkungan A melalui prosedur *recovery* terstandarisasi. Skrip dikembangkan secara paralel dengan perancangan dan implementasi lingkungan agar siap digunakan saat eksperimen dimulai.

5. Persiapan Baseline & Kalibrasi (September – Oktober 2026)

Sebelum eksperimen dimulai, fase kalibrasi dilakukan untuk memastikan validitas internal. Kegiatan meliputi: (a) deployment InvenioRDM pada kedua lingkungan; (b) verifikasi baseline bersih; (c) pengukuran awal konsistensi; (d) verifikasi penggunaan sumber daya node di bawah 70%; dan (e) uji coba awal untuk memvalidasi protokol pengukuran dan skrip otomasi.

6. Eksperimen dan Pengumpulan Data (Oktober – November 2026)

Pelaksanaan tiga eksperimen sesuai desain pada Bab III. E1: konsistensi deployment ($n = 5$ per skenario). E2: *detection time*, *recovery time*, dan *root cause identification time* ($n = 10$ per skenario). E3: *traceability* operasional (deskriptif, dokumentasi kelengkapan audit log per skenario). Data dikumpulkan dalam bentuk *timestamp*, status aplikasi, dan log. Skenario diacak untuk setiap

lingkungan untuk memitigasi *order effect*. Seluruh prosedur dijalankan menggunakan skrip otomatisasi untuk konsistensi.

7. Analisis Statistik dan Interpretasi (November – Desember 2026)

Analisis data meliputi: statistik deskriptif (mean, median, rentang, deviasi standar), uji perbandingan (*Mann–Whitney U test*), perhitungan ukuran efek (Cohen's *d*), dan interpretasi hasil. Pembahasan mencakup identifikasi skenario drift yang paling berdampak, faktor yang mempengaruhi hasil, serta ancaman terhadap validitas internal dan eksternal penelitian.

8. Penulisan Laporan (Juni 2026 – Januari 2027)

Penulisan laporan dilakukan secara paralel sepanjang penelitian. Setiap tahap menghasilkan bab atau bagian yang sesuai. Draft proposal diselesaikan pada pertengahan semester, draft skripsi lengkap pada akhir penelitian.

9. Seminar / Sidang (Januari 2027)

Presentasi dan sidang tugas akhir di depan dosen penguji, mencakup paparan hasil eksperimen, analisis statistik, dan kesimpulan.

DAFTAR PUSTAKA

- Argo Project. Argocd documentation, 2024. URL <https://argo-cd.readthedocs.io>.
- Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and kubernetes. *ACM Queue*, 14(1):70–93, 2016.
- CERN. Inveniordm documentation, 2024. URL <https://inveniordm.docs.cern.ch>.
- K. Chelakis. Creating an open archival information system compliant archive for cern. Master’s thesis, CERN, 2022.
- Cloud Native Computing Foundation. Cncf cloud native definition v1.0, 2024. URL <https://www.cncf.io/about/who-we-are>.
- CNCF GitOps Working Group. Opengitops principles v1.0.0. <https://opengitops.dev>, 2021.
- Jacob Cohen. *Statistical Power Analysis for the Behavioral Sciences*. Lawrence Erlbaum Associates, 2nd edition, 1988.
- Thomas D. Cook and Donald T. Campbell. *Quasi-Experimentation: Design and Analysis Issues for Field Settings*. Houghton Mifflin, 1979.
- M. D’Amore. Gitops and argocd: Continuous deployment and maintenance of a full stack application in a hybrid cloud kubernetes environment. Master’s thesis, Politecnico di Torino, 2021.
- J. D. Fernández, P. Fokianos, P. Herterich, and T. Šimko. Cern analysis preservation: A novel digital library service to enable reusable and reproducible research. In *Research and Advanced Technology for Digital Libraries*. Springer, 2016.
- Thomas A. Limoncelli, Strata R. Chalup, and Christina J. Hogan. *The Practice of Cloud System Administration: Designing and Operating Large Distributed Systems*. Addison-Wesley Professional, 2nd edition, 2018.
- M. E. Matubber. Managing 5g kubernetes infrastructures using gitops principles. Master’s thesis, KTH Royal Institute of Technology, 2025.

- P. Kagganti Nataraja. A security-centric analysis of declarative and imperative deployment approaches in kubernetes-based application environments. Master's thesis, National College of Ireland, 2025.
- E. Paavola. Managing multiple applications on kubernetes using gitops principles. Master's thesis, Turku University of Applied Sciences, 2021.
- O. Pohjola. Mitigating configuration drift in infrastructure-as-code systems. Master's thesis, Aalto University, 2025.
- K. Przerwa and S. Rohr. A modern open source integrated library system by invenio. In *International Conference on Theory and Practice of Digital Libraries (TPDL)*. Springer, 2025.
- A. Rahman, S. I. Shamim, D. B. Bose, et al. Security misconfigurations in open source kubernetes manifests: An empirical study. *ACM Transactions on Software Engineering and Methodology*, 2023.
- R. Shrestha and A. A. N. Ali. Configuration management in kubernetes environments: A gitops approach. In *2024 IEEE/ACM 17th International Conference on Utility and Cloud Computing (UCC)*. IEEE, 2024.
- Ervina Utami and Hanif Al Fatta. Analysis on the use of declarative and pull-based deployment models on gitops using argo cd. In *2021 4th International Conference on Information and Communications Technology (ICOIACT)*. IEEE, 2021.
- Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in Software Engineering*. Springer, 2nd edition, 2012.
- Billy Yuen, Alexander Matyushentsev, Jesse Suen, and Thomas Ekenstam. *GitOps and Kubernetes: Continuous Deployment with Argo CD, Jenkins X, and Flux*. O'Reilly Media, 2021. ISBN 978-1492094877.
- Y. Zhang, U. Paul, M. d'Amorim, and A. Rahman. Configuration defects in kubernetes. *IEEE Transactions on Software Engineering*, 2026.